

## 1. Introdução

Este sistema foi desenvolvido em **Python** utilizando **Streamlit** para criar uma aplicação web de controle de estoque. O sistema permite registrar compras, vendas e despesas, além de gerar indicadores financeiros e gráficos para análise.

Nesta versão do sistema foi implementado **controle de autenticação de usuários** utilizando a biblioteca `streamlit-authenticator`, aumentando a segurança do acesso.

Principais funcionalidades: - Login de usuários - Cadastro de movimentações (compra e venda) - Exclusão de movimentações - Cadastro de despesas - Exclusão de despesas - Controle automático de estoque - Cálculo de custo médio - Dashboard com métricas - Gráficos de vendas - Relatório financeiro (DRE)

Os dados são armazenados em arquivos Excel: - `TabelaEstoque.xlsx` - `Despesas.xlsx`

---

## 2. Tecnologias Utilizadas

| Tecnologia              | Função                          |
|-------------------------|---------------------------------|
| Python                  | Linguagem principal do sistema  |
| Streamlit               | Interface web interativa        |
| Streamlit Authenticator | Sistema de login e autenticação |
| Pandas                  | Manipulação de dados            |
| Altair                  | Criação de gráficos             |
| OpenPyXL                | Manipulação de arquivos Excel   |

---

## 3. Sistema de Autenticação

O login é realizado utilizando a biblioteca **`streamlit_authenticator`**.

### Definição dos usuários

```
credentials = {
    "usernames": {
        "admin": {"name": "Administrador", "password": "1234"},
        "marcel": {"name": "Marcel", "password": "1234"},
    },
}
```

```
        "matheus": {"name": "Matheus", "password": "1234"}
    }
}
```

### Inicialização do autenticador

```
authenticator = stauth.Authenticate(
    credentials,
    "estoque_cookie",
    "abcdef123456789",
    cookie_expiry_days=30
)
```

### Tela de login

```
authenticator.login(location="main")
```

O sistema valida automaticamente se o usuário está autenticado:

```
authentication_status = st.session_state.get("authentication_status")
```

Caso o login esteja incorreto ou não preenchido, o sistema bloqueia o acesso.

---

## 4. Configuração da Aplicação

A aplicação utiliza layout expandido:

```
st.set_page_config(layout="wide")
```

Arquivo principal de armazenamento do estoque:

```
arquivo = "TabelaEstoque.xlsx"
```

---

## 5. Formatação de Valores em Real

Função criada para formatar valores monetários no padrão brasileiro:

```
def moeda_br(valor):
    return f"R$ {valor:,.2f}".replace(",", "X").replace(".",
    ",").replace("X", ".")
```

---

## 6. Cadastro de Movimentação

O cadastro de movimentação permite registrar **compras e vendas** de produtos.

A interface é exibida em um **modal** utilizando:

```
@st.dialog("Cadastrar Movimentação")
```

Campos disponíveis:

```
produto_mov = st.selectbox("Produto", produtos_cadastrados)
tipo_mov = st.selectbox("Tipo", ["Compra", "Venda"])
quantidade_mov = st.number_input("Quantidade", min_value=1)
preco_mov = st.number_input("Preço Unitário", min_value=0.0)
```

### Cálculo do valor total

```
valor_total = quantidade_mov * preco_mov
```

A nova movimentação é adicionada ao DataFrame:

```
df = pd.concat([df, nova_linha], ignore_index=True)
```

Após salvar, o sistema recalcula automaticamente o estoque.

---

## 7. Cálculo Automático do Estoque

O sistema recalcula o estoque de cada produto com base nas movimentações registradas.

Função responsável:

```
def recalcular_estoque(df):
```

A lógica considera:

### Compra

```
novo_estoque = estoque_ant + qtd
```

### Venda

```
novo_estoque = estoque_ant - qtd
```

### Cálculo do custo médio

```
custo_medio = (
    (custo_ant * estoque_ant + valor) / novo_estoque
    if novo_estoque != 0 else 0
)
```

Esse método garante que o valor médio do estoque seja atualizado automaticamente.

---

## 8. Exclusão de Movimentações

O sistema permite excluir registros de movimentação.

```
@st.dialog("Excluir Movimentação")
```

Seleção da movimentação:

```
movimento = st.selectbox(
    "Selecione a movimentação",
    df["MovID"]
)
```

Após excluir, o sistema executa novamente:

```
df = recalculiar_estoque(df)
```

---

## 9. Cadastro de Despesas

O sistema permite registrar despesas operacionais.

Campos disponíveis:

```
descricao = st.text_input("Descrição")
categoria = st.selectbox("Categoria", ["Aluguel", "Fornecedor",
    "Salário", "Marketing", "Outros"])
valor = st.number_input("Valor", min_value=0.0)
```

Os dados são salvos em:

Despesas.xlsx

---

## 10. Exclusão de Despesas

Também é possível remover despesas registradas.

```
@st.dialog("Excluir Despesa")
```

O usuário seleciona a despesa e o sistema remove o registro do arquivo Excel.

---

## 11. Menu Lateral (Sidebar)

A barra lateral permite acessar rapidamente as funcionalidades do sistema.

```
st.sidebar.button("+ Cadastrar Movimentação")
st.sidebar.button("🗑 Excluir Movimentação")
st.sidebar.button("💰 Cadastrar Despesa")
st.sidebar.button("🗑 Excluir Despesa")
```

Também exibe o usuário logado:

```
st.sidebar.write(f"👤 Usuário: {name}")
```

---

## 12. Dashboard de Indicadores

O sistema apresenta métricas importantes de estoque e vendas.

### Quantidade vendida

```
qtd_vendida = vendas["Quantidade"].sum()
```

### Quantidade comprada

```
qtd_comprada = compras["Quantidade"].sum()
```

### Valor vendido

```
valor_vendido = vendas["Valor Total"].sum()
```

Esses valores são exibidos utilizando:

```
st.metric()
```

---

## 13. Tabela de Movimentações

O sistema exibe todas as movimentações de estoque em formato de tabela.

Antes de exibir os dados, os valores são formatados:

```
df_view["Preço Unitário"] = df_view["Preço Unitário"].apply(moeda_br)
```

A tabela é exibida utilizando:

```
st.dataframe(df_view, use_container_width=True)
```

---

## 14. Gráficos de Vendas

O sistema gera gráficos utilizando **Altair**.

### Quantidade vendida por produto

```
alt.Chart(vendas_produto).mark_bar().encode(
    x="Produto:N",
    y="Quantidade:Q"
)
```

### Valor vendido por produto

```
alt.Chart(vendas_produto).mark_bar().encode(
    x="Produto:N",
    y="Valor Total:Q"
)
```

---

## 15. Controle de Despesas

As despesas são exibidas em formato de tabela.

```
st.dataframe(despesas_view)
```

Também é gerado um gráfico por categoria:

```
alt.Chart(despesas_categoria).mark_bar().encode(
    x="Categoria:N",
    y="Valor:Q"
)
```

---

## 16. DRE (Demonstrativo de Resultados)

O sistema calcula automaticamente o resultado financeiro.

### Receita

```
receita_total = valor_vendido
```

### Custo das mercadorias

```
cmv = valor_comprado
```

### Lucro bruto

```
lucro_bruto = receita_total - cmv
```

### Lucro líquido

```
lucro_liquido = lucro_bruto - despesa_total
```

Esses indicadores são exibidos com:

```
st.metric()
```

---

## 17. Conclusão

O sistema desenvolvido permite realizar **controle completo de estoque, vendas e despesas**, além de fornecer indicadores financeiros importantes.

Principais vantagens do sistema:

- Interface simples e intuitiva
- Armazenamento em Excel
- Cálculo automático de estoque
- Controle de custo médio
- Dashboard com gráficos
- Sistema de login seguro

Esse tipo de aplicação pode ser utilizado por **pequenas empresas, comércios ou projetos acadêmicos** para demonstrar conceitos de gestão de estoque e análise financeira.